# Anexo al Examen 1: análisis formal complementario

Héctor Manuel Salazar Rivera
Código 1004735481

9 de noviembre de 2026

## Propósito

Este anexo completa el análisis formal del Examen 1, que está en la misma carpeta. Cubre lo que el examen no trata: la terminación de los dos algoritmos, la corrección de Quicksort, el efecto del pivote por mediana de 3 y dos ajustes al texto del examen. Todo se refiere a la implementación entregada en `section1_comparison/activity2_implementation/algorithms.py`. Las cifras de comparaciones vienen de `section3_deliverables/data/operation_counts.csv`.

## 1. Bubble Sort con salida temprana

La implementación usa una bandera que detiene el algoritmo cuando una pasada no hace ningún intercambio:

```
1: función BubbleSort(A)
2:     n ← |A|
3:     para fin = n − 1 hasta 1 hacer
4:         hubo ← falso
5:         para i = 0 hasta fin − 1 hacer
6:             si A[i] > A[i + 1] entonces
7:                 intercambiar A[i] y A[i + 1]; hubo ← verdadero
8:         si no hubo entonces retornar
```

**Ajuste al mejor caso del examen.** El pseudocódigo del examen no tiene esa bandera. Sin ella, el ciclo externo hace siempre sus $n-1$ pasadas, y un arreglo ya ordenado cuesta también $n(n-1)/2$ comparaciones, es decir $\Theta(n^2)$. El mejor caso $T(n) = n-1$ del examen solo vale con la bandera: en un arreglo ordenado la primera pasada compara $n-1$ pares, no intercambia nada y el algoritmo termina. La medición lo confirma: con $n = 4000$ ordenado hubo 3999 comparaciones. El peor caso no cambia; con $n = 4000$ en orden inverso hubo $7\,998\,000 = 4000 \cdot 3999/2$ comparaciones.

**Corrección de la salida temprana.** Se usa el invariante del examen en la forma de esta implementación: al empezar la pasada con límite *fin*, el bloque $A[\mathit{fin}+1 \ldots n-1]$ contiene los $n-1-\mathit{fin}$ mayores elementos, ya ordenados. Si una pasada no intercambia nada, entonces $A[i] \le A[i+1]$ para todo $i < \mathit{fin}$, así que $A[0 \ldots \mathit{fin}]$ está ordenado. Por el invariante, todos sus elementos son menores o iguales que $A[\mathit{fin}+1]$. Luego el arreglo completo está ordenado y detenerse es correcto.

**Terminación.** Los dos ciclos recorren rangos fijos que no dependen de los datos. El cuerpo interno se ejecuta a lo sumo $\sum_{fin=1}^{n-1} fin = n(n-1)/2$ veces, y cada ejecución cuesta tiempo constante. Por tanto el algoritmo termina para toda entrada.

## 2. Corrección y terminación de Quicksort

El examen analiza un Quicksort que toma el último elemento como pivote. La implementación usa el pivote por mediana de 3 y una partición de dos punteros al estilo de Hoare. Esta partición no deja el pivote en su lugar final; divide el rango en dos bloques y llama a la recursión sobre cada uno:

```
1:  función QUICKSORT(A, lo, hi)
2:      si lo ≥ hi entonces retornar
3:      p ← A[MEDIANADE3(A, lo, hi)]
4:      i ← lo; j ← hi
5:      mientras i ≤ j hacer
6:          mientras A[i] < p hacer i ← i + 1
7:          mientras A[j] > p hacer j ← j − 1
8:          si i ≤ j entonces
9:              intercambiar A[i] y A[j]; i ← i + 1; j ← j − 1
10:     QUICKSORT(A, lo, j)
11:     QUICKSORT(A, i, hi)
```

**Lema 1** (Los punteros no salen del rango)**.** *Cada recorrido interno se detiene dentro de* $[lo, hi]$*.*

*Demostración.* En la primera iteración, $p = A[m]$ para algún $m \in [lo, hi]$. El recorrido de $i$ se detiene a más tardar en $m$, porque $A[m] < p$ es falso, y el de $j$ se detiene a más tardar en $m$ por la misma razón. Tras un intercambio en las posiciones $i \le j$ quedan $A[i] \le p$ y $A[j] \ge p$, porque cada recorrido se detuvo en un valor del lado contrario. Después $i$ avanza a $i+1$ y $j$ retrocede a $j-1$. En la siguiente iteración el recorrido de $i$ se detiene a más tardar en la antigua posición de $j$, donde hay un valor $\ge p$, y el de $j$ a más tardar en la antigua posición de $i$, donde hay un valor $\le p$. Ambas posiciones están en $[lo, hi]$. □

**Lema 2** (Invariante de la partición)**.** *Al empezar cada iteración del ciclo externo se cumple que todo elemento de* $A[lo \dots i-1]$ *es* $\le p$ *y todo elemento de* $A[j+1 \dots hi]$ *es* $\ge p$*.*

*Demostración. Inicialización:* con $i = lo$ y $j = hi$ los dos bloques están vacíos. *Mantenimiento:* el recorrido de $i$ solo pasa sobre valores $< p$ y el de $j$ solo sobre valores $> p$, así que los bloques crecen sin romper la condición. El intercambio toca solo las posiciones $i$ y $j$, que no pertenecen a ninguno de los dos bloques, y deja $A[i] \le p$ y $A[j] \ge p$. Al avanzar $i$ y retroceder $j$, esas posiciones entran a su bloque y la condición se mantiene. *Finalización:* el ciclo termina con $i > j$. Entonces $A[lo \dots j] \le p$ y $A[i \dots hi] \ge p$. Las posiciones entre $j+1$ e $i-1$, si las hay, están en los dos bloques a la vez, así que valen exactamente $p$ y ya están en su lugar. □

**Terminación de la partición.** Por el Lema 1, cada recorrido interno hace un número finito de pasos. Cada iteración del ciclo externo termina el ciclo o aumenta $i$ y reduce $j$, así que $j - i$ baja al menos en 2 por iteración y el ciclo acaba.

**Los subproblemas son menores.** En la primera iteración los recorridos se detienen con $i \leq m \leq j$, así que siempre se ejecuta el paso de intercambio. Al final quedan $i \geq lo+1$ y $j \leq hi-1$, y cada llamada recursiva recibe a lo sumo $n-1$ elementos, con $n = hi - lo + 1$.

**Proposición 1.** *Quicksort termina y deja $A[lo \dots hi]$ ordenado, como una permutación de los valores de entrada.*

*Demostración.* Por inducción fuerte en $n = hi - lo + 1$. Si $n \leq 1$, el rango ya está ordenado y la función retorna. Si $n \geq 2$, la partición termina y solo intercambia elementos, así que el rango sigue siendo una permutación de la entrada. Por el Lema 2 el rango queda en tres bloques: $A[lo \dots j] \leq p$, luego valores iguales a $p$, luego $A[i \dots hi] \geq p$. Las dos llamadas recursivas reciben menos de $n$ elementos, así que por hipótesis terminan y ordenan su bloque sin mover elementos fuera de él. Un bloque ordenado de valores $\leq p$, seguido de valores iguales a $p$ y de un bloque ordenado de valores $\geq p$, forma un rango ordenado. □

## 3. El pivote por mediana de 3

**Costo.** La función `median_of_three` compara $A[lo]$, $A[mid]$ y $A[hi]$, con $mid = \lfloor (lo+hi)/2 \rfloor$, y devuelve la posición del valor del medio. Hace 2 comparaciones cuando las dos primeras bastan y 3 en los demás casos. Es un costo constante por llamada, así que la recurrencia general del examen queda $T(n) = T(k) + T(n-k') + cn + O(1)$, con el mismo orden de crecimiento.

**Arreglo ordenado.** Los tres candidatos están en orden creciente, así que el pivote es $A[mid]$, la mediana exacta del rango. Los punteros se encuentran en $mid$ sin intercambiar nada, y la recursión sigue sobre dos mitades que siguen ordenadas. Entonces $T(n) = 2T(n/2) + cn$ y, como en el mejor caso del examen, $T(n) = \Theta(n \log n)$. El peor caso del examen, un arreglo ordenado con el último elemento como pivote, ya no se da con esta implementación. Medido con $n = 4000$: 48 105 comparaciones y profundidad de recursión 12, cerca de $\log_2 4000 \approx 11{,}97$.

**Arreglo inverso.** Los candidatos están en orden decreciente y el pivote vuelve a ser la mediana exacta. La primera partición intercambia los pares simétricos $(lo, hi), (lo+1, hi-1), \dots$ y deja todo el rango ordenado. Desde ahí el algoritmo sigue como en el caso ordenado, con $\Theta(n \log n)$. Medido con $n = 4000$: $2000 = n/2$ intercambios en total y profundidad 12.

**Valores repetidos.** Los recorridos usan comparaciones estrictas ($<$ y $>$), así que se detienen en los valores iguales al pivote y los intercambian. Con muchos valores iguales los punteros se encuentran cerca del centro y las particiones siguen equilibradas. Medido con $n = 4000$ y solo 10 valores distintos: profundidad entre 18 y 20.

**Peor caso.** La mediana de 3 no elimina el caso cuadrático. El experimento construye una entrada con el adversario de McIlroy (1999), en la función `median_of_three_killer` de `section1_comparison/activity3_experiment/experiment.py`. Con esa entrada, cada partición separa solo unos dos elementos del resto, y la recurrencia queda

$$T(n) = T(n-2) + cn + O(1) \quad \Longrightarrow \quad T(n) \approx c\,(n + (n-2) + (n-4) + \cdots) \approx \frac{c\,n^2}{4} = \Theta(n^2).$$

Medido con $n = 4000$: 4 019 986 comparaciones, cerca de $n^2/4 = 4\,000\,000$, y profundidad $2001 \approx n/2$. Esa profundidad también fija el espacio: la implementación hace las dos llamadas recursivas

sin eliminar la última, así que la pila crece $\Theta(n)$ en este caso. Por eso el experimento sube el límite de recursión de Python.

**Caso promedio.** Este anexo no demuestra el caso promedio. En las entradas aleatorias medidas, las comparaciones crecen como $n \log n$.

## 4. Ajuste de terminología

En la página 4 el examen llama *logarítmica* a la complejidad $\Theta(n \log n)$. Logarítmica es $\Theta(\log n)$, como el espacio de Quicksort en el mejor caso. El término para $\Theta(n \log n)$ es *linealítmica* o *cuasilineal*.

## 5. Resumen

| Algoritmo | Mejor caso | Peor caso | Espacio extra |
|---|---|---|---|
| Bubble Sort con bandera | $\Theta(n)$, ordenado | $\Theta(n^2)$, inverso | $O(1)$ |
| Quicksort, mediana de 3 | $\Theta(n \log n)$, ordenado o inverso | $\Theta(n^2)$, adversario | $\Theta(\log n)$ a $\Theta(n)$ |

## Referencias


- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press.
- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10-16.
- McIlroy, M. D. (1999). A killer adversary for quicksort. *Software: Practice and Experience*, 29(4), 341-344.